Project Milestone P2: Analysis and High-Level Design

Simple File Explorer

Team	Team 04, section .010
Document version	1.0 (baseline for P2; grows from your P1 version 0.x)
Date	10/29/2026
Repository URL	https://github.com/AshtonElliott/SimpleFileExplorer/tree/main/p2
Git tag for this milestone	`p2` (points at the commit that contains this exact PDF)
Submitted on Canvas by	Austin Robson

Team member	NetID	GitHub username
Austin Robson	axr230166	austin-robson
Yukai Ma	dal833810	yukaima
Ashton Elliot	aee230007	AshtonElliott
George Chakiath Dani	gnc230001	gchakiathUTDallas
Rowan Egan	rje240001	
Zachary Gregory	dal465328	ZacG-codes

0 Revision History, Contribution Record and Changes Since P1

Version	Date	Author	Summary of change

0.10	10/18/26	Austin Robson	Added more quality requirements, reviewed and finalized content before submission
1.0	10/30/26	Austin Robson	P2 template translated into markdown for github usage
1.1	10/31/26	George Chakiath Dani	Ported sections 1-3 from P1
1.2	11/3/26	Ashton Elliot	Began work on prototype app
1.3	11/6/26	Austin Robson	Updated section 3 and 5, added sequence diagrams
1.4	11/6/26	George Chakiath Dani	Updated section 7
1.5	11/7/26	George Chakiath Dani	Updated section 8,9, added spike s-01
1.6	11/7/26	Yukai Ma	Updated section 6, added Component and Deployment diagrams, updated use case diagram
1.7	11/7/26	Austin Robson	Added spike s-02
1.8	11/8/26	Austin Robson	Updated spike s-01
1.9	11/8/26	Austin Robson	Finalizing document sections and updated domain model
2.0	11/8/26	Ashton Elliott	Finalized document and converted to PDF.

0.1 Individual contribution record (P2)

Team member	**Sections and artifacts this person produced for P2**	**Evidence in the repository (commit hashes, PR numbers)**	**Reviews given (PR numbers)**
Austin Robson	Added .md for P2 document	Commit c7bac05	N/A

Team member	Sections and artifacts this person produced for P2	Evidence in the repository (commit hashes, PR numbers)	Reviews given (PR numbers)
George Chakiath Dani	Prepared Section 4 - Use-Cases Analysis, using information and requirements indemnified in P1	497108a, a1091e7	No PR reviews as of 10/31/26.
George Chakiath Dani	Updated Sections 1-3 (Introduction, Overall Description, and Requirements) from P1.	Commits: 4bea388, e8b085e, 5fa1711, 2ff27a7, 6383af3, e032a95, 78242bf	No PR reviews as of 10/31/26.
Austin Robson	Updated formatting for information imported from P1	Commit 12a3fe3	N/A
Ashton Elliott	Created basic JAVA GUI as a base to start from	Commit 9a5e323	No PR reviews as of 11/3/26.
Ashton Elliott	added GUI File Search	Commit 82f3d2d	No PR reviews as of 11/4/26.
Austin Robson	Updated use cases in section 3	Commit 55508fc	N/A
Austin Robson	Added domain model	Commit 937c8f4	N/A
Ashton Elliot	Implemented Domain Model PNG to P2 Report	Commit 6359ff7	N/A
Austin Robson	Updated section 5 Analysis models	Commit cb6eaf	N/A
George Chakiath Dani	Finished up Section 7.1 - Architecture Decision Records (ADRs 01-03); Section 7.2 - Trade-off Summary	Commits: 273fb19, 86deee0, dcf7f83, 24d4394	No PR reviews as of 11/6/26
Austin Robson	Updated section 5 and added sequence diagrams	Commit 5b8800e0	N/A

Team member	Sections and artifacts this person produced for P2	Evidence in the repository (commit hashes, PR numbers)	Reviews given (PR numbers)
George Chakiath Dani	Section 8 - Prototype Evidence: Architecture Spike (S-01: ls parsing)	30f03ac	No PR reviews as of 11/7/26
George Chakiath Dani	Section 9.1 - Extended Traceability Matrix: Section 9.2 - Gap Analysis	Commits: 2a64472, 9f791bc, e6cb1c7	No PR reviews as of 11/7/26
Austin Robson	Updated section 5	Commit 2f81e8f	N/A
Yukai Ma	Finish Section 6, remade some previous diagrams,come out some suggestions for modifications	Commits: the merge history, 8470ba3, 6613443	No PR reviews as of 11/8/2026
Austin Robson	Updated diagrams, added spike images	Commit 3b30dc6	N/A
George Chakiath Dani	Fixed Section 7: Design Rationale, documenting ADR-01 through ADR-04 and the Section 7.2 trade-off summary.	Commit: 23e4264	No PR reviews as of 11/8/2026
George Chakiath Dani	Fixed Section 8: Prototype Evidence, documenting spikes S-01 (ls parsing)	Commit: 123e82e	No PR reviews as of 11/8/2026
George Chakiath Dani	Fixed Section 9: Traceability Matrix and Gap Analysis	Commit 5d788ea	No PR reviews as of 11/8/2026
George Chakiath Dani	Updated Section 0.2 - Changes since the P1 baseline, documenting CH-01 through CH03.	Commit: e5bc4b4	No PR reviews as of 11/8/26

Team member	Sections and artifacts this person produced for P2	Evidence in the repository (commit hashes, PR numbers)	Reviews given (PR numbers)
George Chakiath Dani	Updated Section 10.2 - Plan to P3, documenting the milestones, owners, dates, and completion criteria for P3.	Commit: 741ceba	No PR reviews as of 11/8/26
George Chakiath Dani	Updated Section 12 - Baseline v1.0 Declaration and team signature table.	Commit: e36a14e	No PR reviews as of 11/8/26
George Chakiath Dani	Added the Context Diagram to Section 2 - Overall Description.	Commit: 9791fe6	No PR reviews as of 11/8/26
Zachary Gregory	Updated formatting in 6.3 table removed empty rows	Commit: fbd631a	No PR reviews as of 11/8/26
Austin Robson	Updated Spike S-01, updated information relying on S-01, updated P2 readme	Commit: df76320	N/A
Ashton Elliott	Merged Yukai's Work to Main.	Commit: 40b06a3	N/A
Austin Robson	Updates to readme and domain diagram	Commit 8670599	N/A
Ashton Elliott	Converted Markdown to PDF.	Commit 1072ab7	N/A

0.2 Changes since the P1 baseline

Change ID	What changed (IDs affected)	Why (P1 feedback, new finding, spike result)	Knock-on effect	Agreed on
CH-01	Added spike S-02 comparing Windows File	The comparison recorded Windows File Explorer at 39 seconds	Supports the decision to use	11/7/26

Change ID	What changed (IDs affected)	Why (P1 feedback, new finding, spike result)	Knock-on effect	Agreed on
	Explorer and WSL search performance.	and WSL at 12 seconds, with 14 versus 223 results.	command line tools in ADR-04.	
CH-02	Added new risk R6 concerning whether Linux command output can be converted into data usable by the GUI.	Identified an additional integration risk during P2 design.	Added a mitigation task for Ashton	11/7/26
CH-03	Added architectural decisions ADR-01 through ADR-04, covering MVC, local Java/command execution, read-only access, and command line tools.	Limiting file access to linux commands only was found to be difficult due to how linux commands display file info. P2 design work clarified how the application will meet its functional requirements and constraints through other means such as Java or powershell.	App will require different dependencies to run. Established architecture and component responsibilities in Sections 6 and 7.	11/8/26

1 to 3 Introduction, Overall Description and Requirements (carried forward)

1 Introduction

1.1 [Purpose of this document and the system]

Simple File Explorer (SFE) is a slimmed down reimagining of the stock file explorer on Windows, reducing lag and long loading times by leveraging linux commands in a simple GUI.

1.2 [Scope]

In scope	Out of scope
GUI	Drive manager
File probing	Settings
Quick access sidebar	Networking
Searching	File movement

Why the exclusions were excluded: Drive management and file movement outside of the dedicated operating system methods is tricky and potentially dangerous, and therefore out of scope for this file reader app. Network connectivity introduces vulnerabilities, and there are no options to necessitate a settings menu.

1.3 [Definitions, acronyms, and abbreviations

Term or abbreviation	Meaning as used in this project
SFE	Simplified File Explorer; Name of app
WSL	Windows Subsystem Linux; App relies on it for linux use
WFESE	Windows File Explorer Search Engine; Windows File Searcher that comes with all windows computers.

1.4 [References]

1. Windows File Explorer (WFE)
2. Windows Subsystem Linux (WSL)
3. Java
4. Powershell

2 Overall Description

2.1 [Product perspective and context]


Context Diagram

The Simple File Explorer system allows users to search for files through a user interface. The system is responsible for handling user requests, translating them into WSL commands, processing search results, and tracking search history. The End User, Windows Subsystem for Linux (WSL), and Host OS/File System are outside the system boundary because they are not being developed as part of the project. Instead, the Simple File Explorer communicates with these external systems to execute

searches and retrieve file information. This boundary defines what our project is responsible for building and what it relies on externally.

2.2 [Stakeholders and how you reached them]

Stakeholder	Role	What they need	How we learned this
End users	Downloads and uses the app	A file searcher that takes less than 10 minutes to find the desired file from 300,000 files with the right path	We ran a test on Window's File Explorer search engine to see how long it would take to find a file with the keyword "BERT". It took 11 minutes and 9 seconds to find a file with the keyword "BERT", but it was a cached path and not the actual file path.
Software Testers	Tests functionality and performance of app	The app must return the accurate file paths and accurately find files within the required time.	Based on the Windows File Explorer test, which took 11 minutes and 9 seconds and returned a cached path.
Regulator	Check if the app is legal	Need the app to obey the law (e.p protect private information)	To check all related law requirements before programming
Software developers	Design, build and maintain the app	Develop a reliable and efficient file search app that demonstrates fast searching times for each file searched	Based on responsibilities of the development team and our group members
Maintenance Team	Fixes Bugs and updates the app after release	Code that can be easy to maintain, along with useful messages and comments for future debugging.	Based on the need to keep the app working after development

2.3 [User characteristics]

The users are individuals who want to find files within their computer as soon as possible. They should know how to use a search engine and that Windows has a file search engine. What cannot be assumed that they know is the fact that said Windows file search engine is inefficient as it takes 11 minutes and 9 seconds to find a single result.

2.4 [Constraints]

Must run locally Must follow the law to protect users' information and documents. Must use functions and commands to make system calls

2.5 [Assumptions and dependencies]

'#'	Assumption/Dependency	What breaks if it does not hold
A1	WSL installed	The system will not be able to execute Linux comands, and the program will fail
A2	Java installed	The system will not be able to execute the program
A3	Java is able to interface with command line	Java will not be usable for completing this project

2.6 [Feasibility Summary]

Feasibility gate	Verdict	Evidence behind the verdict
Technical	Yes	A small test was conducted within a Linux Virtual Machine where the keyword "BERT" was searched throughout the entire Virtual Machine with grep.
Operational	Yes	From the test, the system does increase the loading speed a lot. The end users who really want fast access to the file will choose to use it.
Schedule	Yes	Through the team discussion, the estimated time is within the deadline.
Economic / resource	Yes	Because this is a small app with no paid external dependencies, the economic cost will be little to none

3 Requirements

3.1 [Functional requirements]

ID	Requirement	Priority	Source	Acceptance
FR-01	The file information displayed in the GUI shall be accurate to its actual data	MUST	Devs	Verify that displayed information matches what is shown by command line tools (ls -l)
FR-02	Folders shall be navigable via mouse or keyboard actions (double click, arrows, enter, backspace, etc)	MUST	Devs	GUI navigates through directories and displays new contents without error
FR-03	Users shall search for strings in directory/file names/contents	MUST	Devs	File searches in the GUI are accurate with results from command line tools
FR-04	Drives, commonly used folders, and folder history shall be displayed in a sidebar	MUST	Devs	SFE includes a sidebar for quick access to drives, common folders (such as Documents, Downloads, etc), and recent folders.
FR-05	Errors shall be reported within the GUI	SHOULD	Devs	Test errors display correctly when triggered and cancels the action
FR-06	Images shall display with thumbnails	COULD	Devs	Images have a smaller version (a thumbnail) generated and cached to display alongside their information
FR-07	Users shall be able to create, delete, and rename files.	WON'T	Devs	Users are able to create/delete/rename files within the GUI
ID	**Requirement**	**Priority**	**Source**	**Acceptance**
FR-08	Users shall be able to move files from one directory to another.	WON'T	Devs	Files are able to be transferred to other directories, by copy-paste or drag 'n' drop, without loss of file/directory sanity.

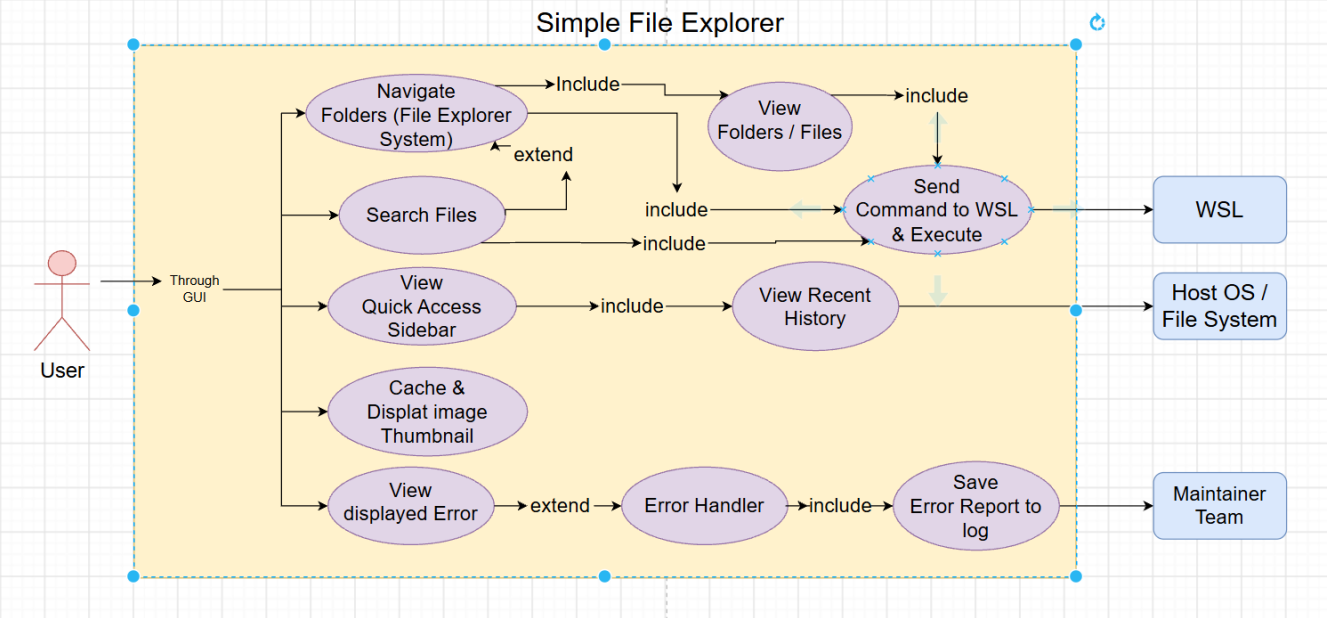
3.2 [Quality requirements]

ID	Requirement	Priority	Source	Verification
QR-01	Searches must complete within 10 minutes of their initialization	MUST	Devs	A tester can confirm this by tracking load times while searching files in a drive filled with 1TB of contents.
QR-02	Entering new folders should load their new content within 5 minutes of their opening	MUST	Devs	A tester can check whether the a folder of 250 children are displayed in time.
QR-03	Search results shall contain 20 results before needing to scroll	SHOULD	Devs	Search results containing >20 entries will create a scroll bar in the window
QR-04	The app shall open within 5 minutes of its starting	SHOULD	Devs	Start times for the app will be measured to find if it successfully boots in under 5 minutes in a folder of 250 children.
QR-05	Errors shall be logged within 1 minute of their occurrence	SHOULD	Maintenance Team	A tester can trigger 10 different errors and verify that each is recorded in the system within 1 minute.

3.3 [Constraint requirements]

ID	Constraint	Source	Why
CON-01	Must run locally	Devs	Running with or through a third-party service creates unnecessary load times and opens up vulnerabilities
CON-02	The system doesn’t write, move or delete any user file	Devs	Read-only access removes the risk of data loss
CON-03	Must use commands or functions to make system calls	Devs	Use Java, Linux, or other command line tools that can achieve fast access to the file in memory

3.4 Use cases



3.5 Traceability

See section 9.1. for the requirements traceability matrix.

---

4 Use-Case Analysis

4.1 Use cases selected for analysis

Use case	Name	Why it was selected	Non-trivial steps
UC-01	View Files	"File" part of file explorer, integral to successfully creating a file viewer	Click on file, run command to get file attributes, GUI displays file attributes
UC-02	View Folders	"Explorer" part of file explorer, needs to allow navigation of directories	Double-click folder, run command to change directory to that folder, refresh GUI display to show folder's contents
UC-03	Search terms	Prove that searching with simpler tools allows for faster results	Accepting a search term, finding matching files and folders, and finally displaying results.

4.2 Expanded use cases

UC-01 View files

Primary actor	**End User**

Supporting actors and external systems	WSL, GUI
TUCBW (this use case begins when)	The user clicks on a file to view its details
TUCEW (this use case ends when)	The user sees the selected file's attributes
Preconditions	The application was initiated successfully. The user has permission to read the file.
Postconditions	The file's attributes are displayed on the GUI and are accurate to command line results.
Traces to requirements	FR-01, CON-02

Actor input / action	**System response**
(what the actor sees before touching anything)	(0) The system displays the contents of the currently selected folder.
(1) The user selects a file.	
	(2) The system finds and displays attributes for the selected file.
(3) The user views the file attributes on the GUI.	

Alternate flows

Alt. flow	**At step**	**Condition**	**What happens**	**Ends or rejoins at**
A1	1	The selected file cannot be accessed because of	The system informs the user that the file couldn't be	Ends

Alt. flow	**At step**	**Condition**	**What happens**	**Ends or rejoins at**

		permissions set by the admin.	accessed. GUI directory remains unchanged.	
A2	1	The selected file is no longer available.	The system informs the user that the file is unavailable.	Ends

UC-02 View folders

Primary actor	**End User**
Supporting actors and external systems	File System
TUCBW (this use case begins when)	The user selects a folder to navigate to
TUCEW (this use case ends when)	The user sees the contents of the new folder
Preconditions	The application was initiated successfully. The user has access to the directories being searched.
Postconditions	The selected folder's contents are displayed accurately. The user can view the contents of the new folder.
Traces to requirements	FR-02, QR-02, CON-01, CON-02

Actor input / action	**System response**
(what the actor sees before touching anything)	(0) The system displays the contents of the currently selected folder.
(1) The user selects a different folder.	
	(2) The system redirects the GUI's current directory to the selected folder.
(3) The user views the contents of the newly selected folder.	

Alternate flows

Alt. flow	At step	Condition	What happens	Ends or rejoins at
A1	1	The selected folder cannot be accessed because of permissions set by the admin.	The system informs the user that the folder couldn't be accessed. The previously displayed folder remains unchanged.	Ends
A2	1	The selected folder is not longer available.	The system informs the user that the folder is unavailable and keeps the previous folder displayed.	Ends

UC-03 Search terms

Primary actor	**End User**
Supporting actors and external systems	File System
TUCBW (this use case begins when)	The user enters a search term to find matching files or folders.
TUCEW (this use case ends when)	The user sees all the search results for their term.
Preconditions	The application has started successfully. The user has access to the application.
Postconditions	Matching files and folders are displayed accurately. The user can view the locations of the matching results.
Traces to requirements	FR-03, QR-01, CON-01, CON-02

Actor input / action	**System response**
(what the actor sees before touching anything)	(0) The system displays the current location and provides the user with the ability to search for files and folders.

Actor input / action	**System response**

(1) The user enters a search term.	
	(2) The system executes a search with the given term and displays files and folders matching the entered term, including thier locations.
(3) The user reviews the displayed search results.	

Alternate flows

Alt. flow	At step	Condition	What happens	Ends or rejoins at
A1	2	No matching files or folders are found	The system informs the user that no matching results were found	Ends
A2	2	Some locations cannot be accessed during the search.	The system informs the user that some results may be unavailable due to access restrictions.	Rejoins at 3

4.3 Self-review log

Use case	Check from the Lesson 13 review checklist	Defect found	How it was fixed
UC-01	Does the use case start with step (0)	The orginal use case didn't explain what the user saw before interacting with the system.	Added step 0, describing the currently displayed folder contents.
UC-02	Does every row hold exactly one turn?	The system search process and displaying results could be described as multiple actions in one step	separated the user's search input from the system's response.
UC-03	Does the use case end with an	The original interaction did not clearly explain how the	Added a final actor step where the user is able to
Use case	**Check from the Lesson 13 review checklist**	**Defect found**	**How it was fixed**

	acknowledgment?	user knows the interaction was finished.	review the displayed contents.

5 Analysis Models

5.1 Domain model

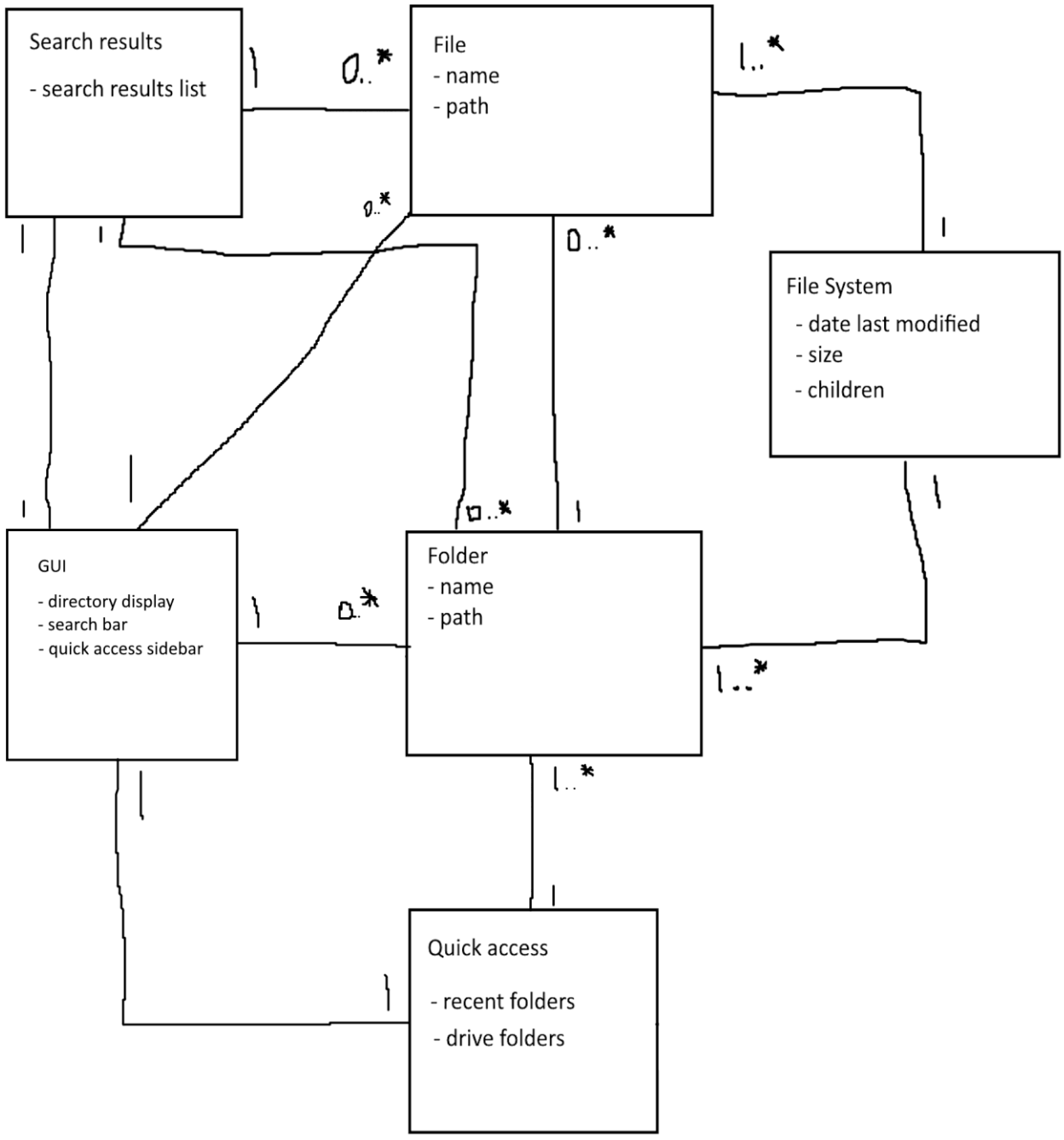


Figure 5.1 Domain model (conceptual classes, attributes, associations with multiplicities)

Class	What it represents (one line)	Key attributes	Associations (with multiplicity)

GUI	How the system displays to the user and how the user interacts with the system	directory display, search bar, quick access sidebar	GUI 1:1 Quick access, GUI 1:1 Search results, GUI 1:0..* Folder, GUI 1:0..* File
File	Individual files in the file system	Name, path	File 1..*:1 File system. File 0..*:1 Folder
Folder	Folders/directories within the file system	Name, path	1 Folder 1..*:1 File system. Folder 1:0..* File
File system	Abstraction of what handles file specifics	Date last modified, size, children	File 1..*:1 File system. Folder 1..*:1 File system
Search results	Files and folders listed after searching	List of search results	Search results 1:0..* File. Search results 1:0..* Folder.
Quick Access	Easy access to drive folders and recent folders	Lists drive folders and recent folders	Quick access 1:1..* Folder

5.2 Object interaction models

5.2.1 UC-01 View Files

(a) Non-trivial steps. (1) The user selects a file. (2) The system finds and displays attributes for the selected file. (3) The user views the file attributes on the GUI.

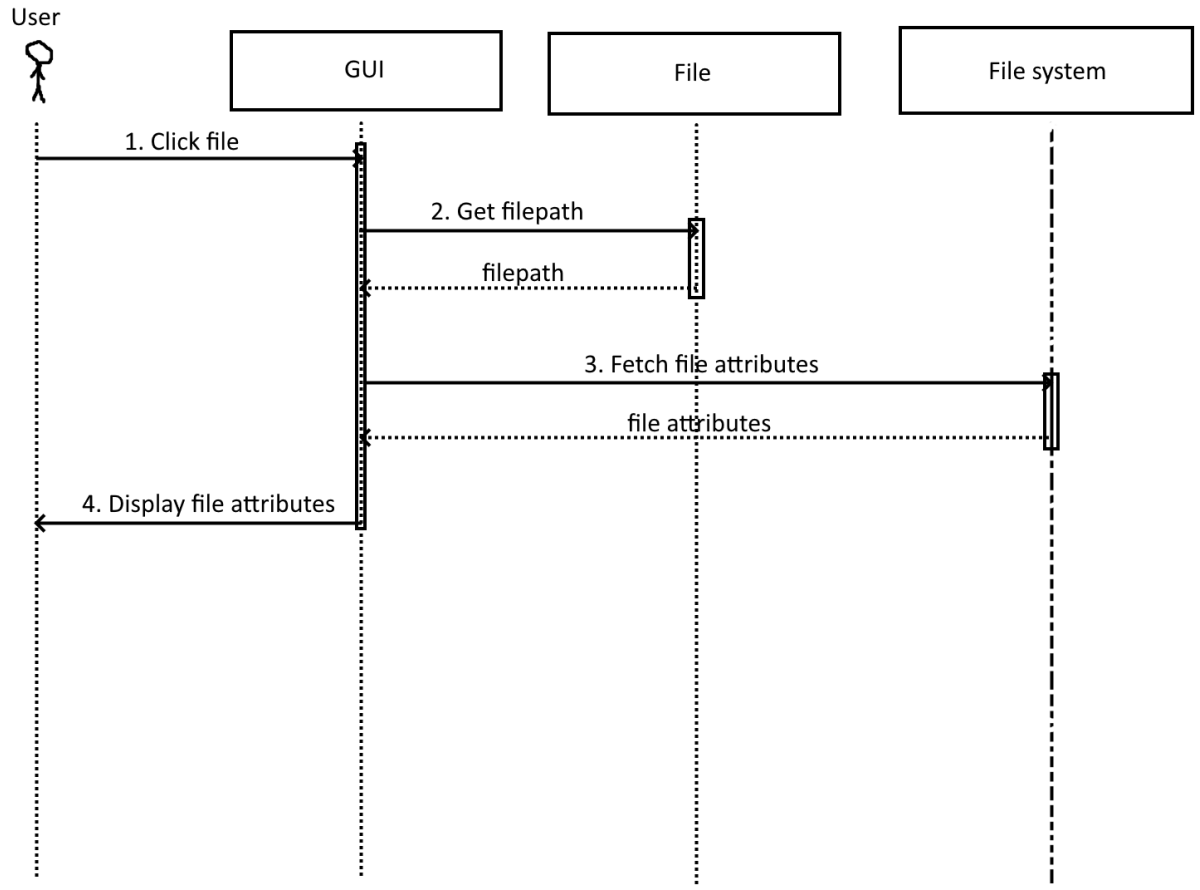
"(0) The system displays the contents of the currently selected folder." is trivial because it is implied as the current state of the program, before interaction takes place.

(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
1	User	Click file	File's path	GUI
2	GUI	Get filepath	File's filepath	File
3	GUI	Fetch file attributes	File's filepath	File system

4	GUI	Display file attributes	File's attributes	User

(c) Analysis sequence diagram.



5.2.2 UC-2 View Folders

(a) Non-trivial steps. (1) The user selects a different folder. (2) The system redirects the GUI's current directory to the selected folder. (3) The user views the contents of the newly selected folder.

"(0) The system displays the contents of the currently selected folder." is trivial because it is the default state before any interaction.

(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
1	User	Double-click folder	folder's path	GUI

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
2	GUI	Fetch folder path	folder's path	Folder
3	GUI	Fetch folder children	folder's path	File system
4	GUI	Change directory	folder's children	GUI
5	GUI	Display contents	folder's children	User

(c) Analysis sequence diagram.

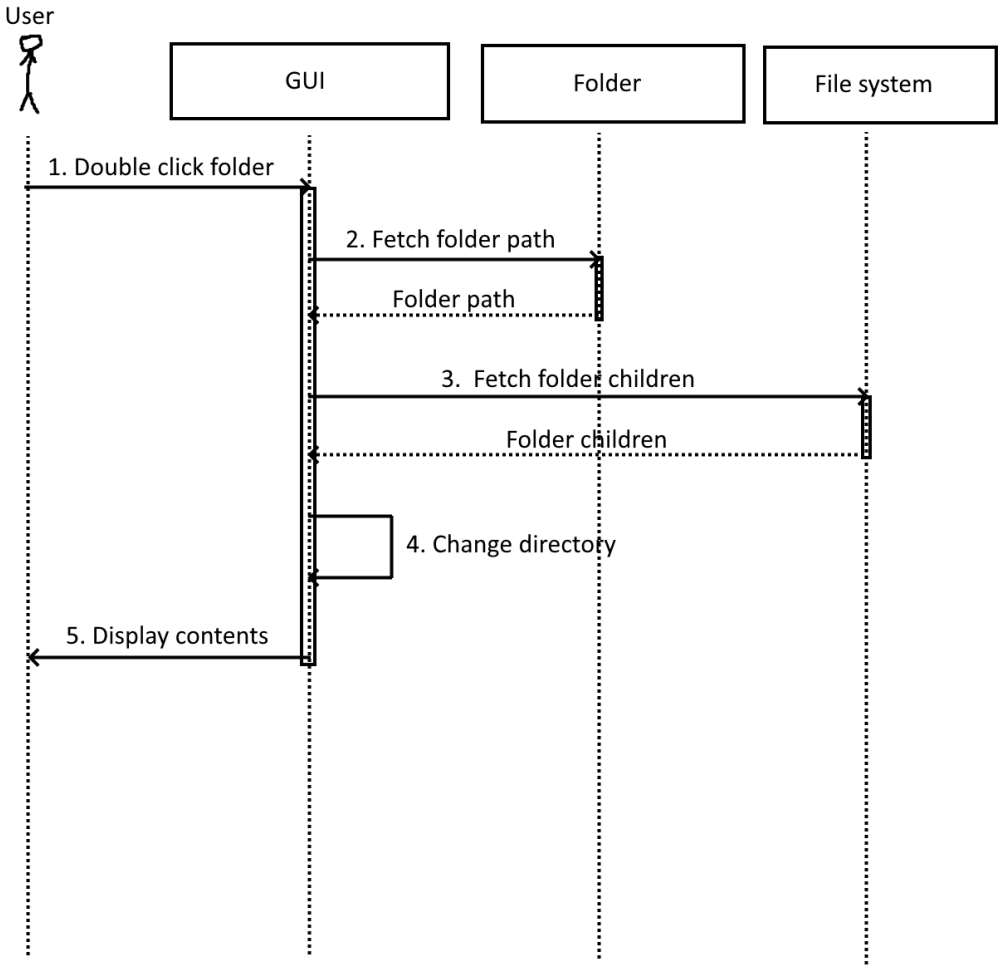


Figure 5.2.2 Analysis sequence diagram for UC-02; message numbers match the scenario table

5.3 Consistency cross-check

Check	Result (pass, or what failed)	Fixed in

K1	Every item in an actor column (4.2) is an attribute in the domain model (5.1)	Pass
K2	Every object on a sequence diagram is a domain class or a declared interaction object	Pass
K3	Every message on a sequence diagram corresponds to exactly one scenario-table row	Pass
K4	Every non-trivial step in 5.2 (a) has at least one scenario-table row	Pass
K5	Every analyzed use case traces to at least one functional requirement	Pass
K6	The last messages of each diagram make the use case postcondition true	Pass

6 High-Level Design (Architecture)

6.1 Architectural drivers: quality-attribute scenarios

QAS	Quality attribute	Stimulus	Artifact	Response	Response measure	From
QAS-01	Performance	When searching for a keyword in a folder which containing 1000 files	Search term	return all files related	the search should finish and return the correct result within 10 min	QR-[01]
QAS-02	Performance	When the user starts this app	GUI/File Explorer	the app starts successfully	Start times for the app will be measured to find if it successfully boots in under 5 minutes	QR-[04]

QAS-03	Availability	When an error occurs	Error handler	Error is logged to the report	trigger 10 different errors and verify that each is recorded in the system within 1 minute.	QR-[05]

6.2 Architectural style and alternatives

Style considered	Drivers it serves	What it costs	Verdict
Model-View-Controller	It separates the actions into three layers, GUI/search/sidebar can be the view; file explorer is the controller; folder, file and WSL commands can be the model. This provide	RecoverabilityNeed carefully designed interface, need keep view update when directory is changed.	**Chosen**
Style considered	**Drivers it serves**	**What it costs**	**Verdict**
	maintainability(separate layer is easy to make changes), Performance,and Security(separate viewer and controller with model)		
pipe-and-filter	Good for dealing with data flow, easier for search	poor performance	Rejected
Eventdriven	allow Asynchronous Operations, good for long-running searching	poor traceability	Rejected

6.3 Component view

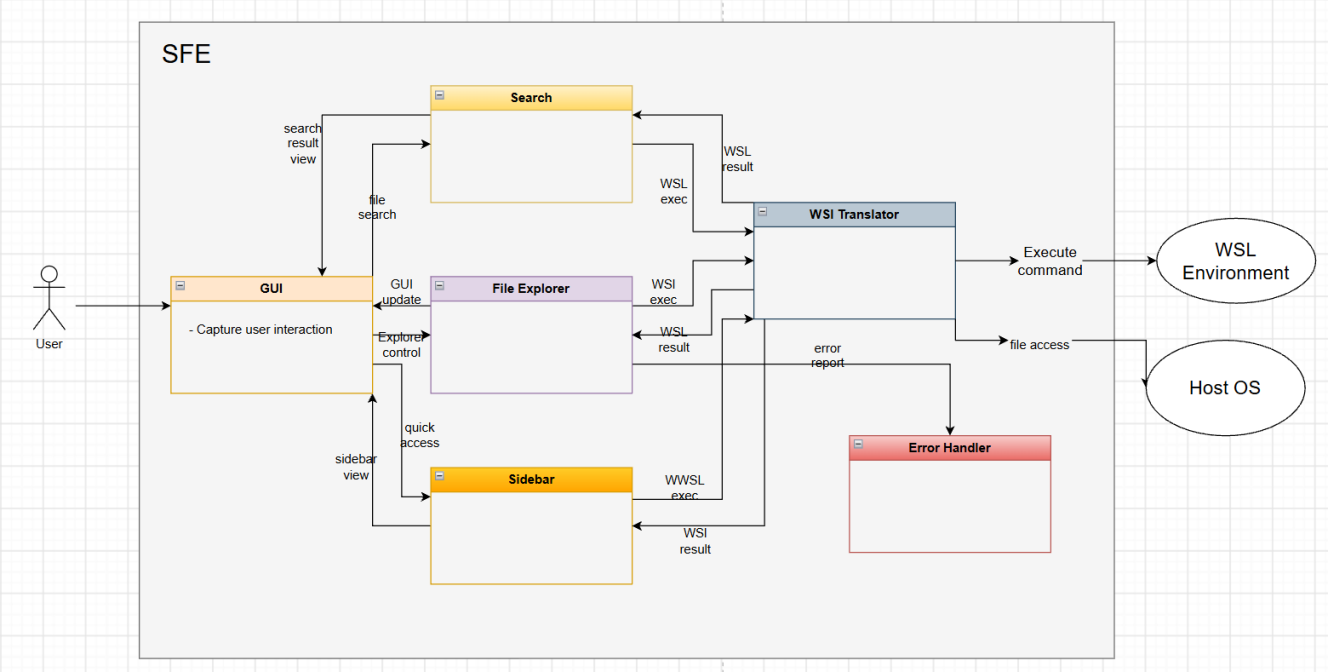


Figure 6.3 Component diagram: components, provided and required interfaces, external systems

Component	Responsibility (one sentence)	Provides	Requires	Answers to (UC / QAS / ADR)
GUI Presentation	Captures user inputs and renders file lists, search results, sidebar links, and preview images on screen.	User Interaction	ExplorerControl, FileSearch, QuickAccess	UC: View Folders/Files, View Quick Access Sidebar , MVC View Component
File Explorer	Orchestrates desktop navigation actions, routes queries to backend sub-engines, and coordinates data flow to the UI.	Explorer Control	WSL Execution, FileAccess, LoggingService	UC: Navigate Folders, View Folders/Files , MVC Controller Component
Search	Filters directory hierarchies and identifies files	FileSearch	FileAccess, LoggingService	UC: Search Files
Component	**Responsibility (one sentence)**	**Provides**	**Requires**	**Answers to (UC / QAS / ADR)**

	matching specified user search criteria.			
Sidebar	Maintains user shortcuts, and recent navigation history for quick access.	Quick Access	File Access	UC: View Quick Access Sidebar, View Recent History
WSL translator	Translates internal requests into POSIX commands and executes them within the Windows Subsystem for Linux.	WSL Execution	External WSL Environment	UC: Send Command to WSL & Execute
Logger & Error Handler	Intercepts application failures, formats error reports, and appends diagnostic entries to persistent logs.	Logging Service	External Log Storage	UC: Display Error, Error Handler, Save Error Report to log

6.4 Deployment view

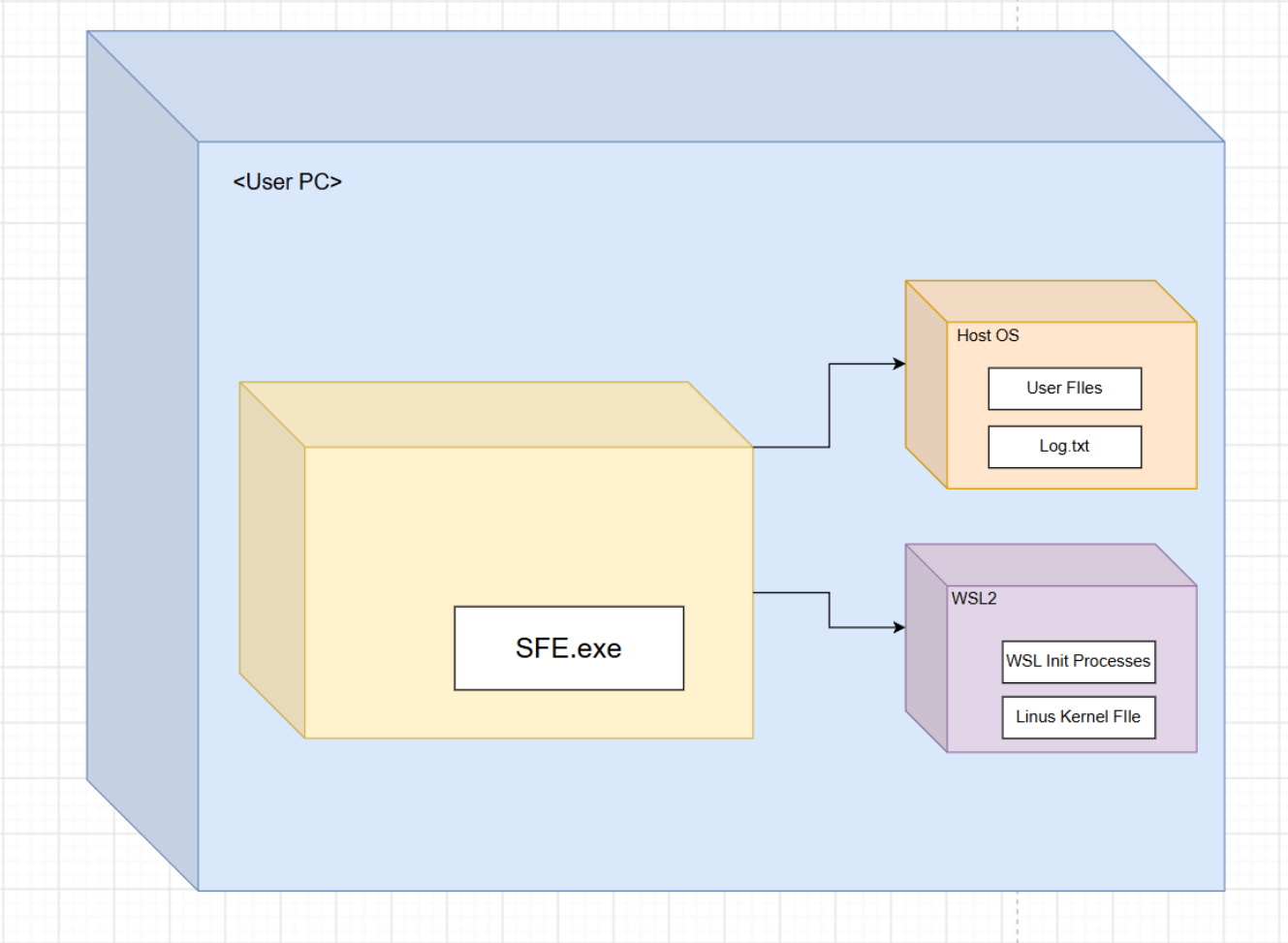


Figure 6.4 UML deployment diagram: nodes, execution environments, deployed artifacts, communication paths with protocols, trust boundaries

Node	What runs on it (artifacts, components)	Communicates with (protocol)	Trust zone
User PC	SFE.exe(GUI, File Explorer, Search, sidebar,etc.)	Host OS, WSL2 environment	Local machine zone
Node	**What runs on it (artifacts, components)**	**Communicates with (protocol)**	**Trust zone**
Host OS	User files, log.txt	SFE.exe	Umanaged data zone, need permit to access
WSL2 environment	Linux Kernel File, WSL Init Processes	SFE.exe	Less trust zone, need permit

6.5 From analysis to architecture

Analysis object (from 5.1 and 5.2)	Component that owns it (6.3)	Note (split, merged, external)

GUI	GUI	N/A
File explorer, file, folder	file explorer	merged, as the file and folder is under control of file explorer
search	search	N/A
sidebar	sidebar	N/A
Error log	Error habdler	N/A
WSL translator	WSL translator	N/A

6.6 Design principles applied

Principle	**Where it shows in your design**	**Evidence (component, interface, ADR)**
Separation of concerns	Separation between file explorer and special logic:search	component: file explorer and search
Information hiding	implement details are hidden behind WSL translator	component: WSL translator
Low coupling	there's no complex stuff passed between components, just basic datatypes(name, path, time last modified, etc.)	Component: file explorer, interface between components
Principle	**Where it shows in your design**	**Evidence (component, interface, ADR)**
High cohesion	merge file and folder into file explorer system	component: file explorer
Design for change	MVC structure allow future replacement for GUI	component: GUI
Keep it simple (KISS)	Using direct interface method calls between local components	interface: explorer control

7 Design Rationale

7.1 Architecture decision records

ADR-01 [Using Model-View-Controller Architecture]

Status	Accepted
Date and owner	11/08/2026 - George
Context (drivers)	The application contains a GUI, file navigation, and searching functionality. These responsibilities need to be separated to improve maintainability and make future changes easier
Options considered	1. Use a Pipe-and-Filter architecture. 2. Use an Event-Driven architecture. 3. Use Model-View-Controller (MVC) architecture.
Decision	Use MVC architecture. The GUI acts as the View, File Explorer acts as the Controller, and file/folder data and WSL operations are handled by the Model.
Consequences	Improves separation of concerns and maintainability because changes to the GUI and backend logic can be made more independently. The accepted cost is additional coordination between components and the need to maintain clear interfaces between the View, Controller, and Model.
Evidence	Section 6.2 identifies MVC as the chosen architectural style. Section 6.3 defines separate GUI Presentation, File Explorer, Search, Sidebar, and WSL Translator.
Related	QAS-01, QAS-02, FR-01, FR-02, FR-03

ADR-02 [Running the Application Locally]

Status	Accepted
Date and owner	11/06/2026 - George

Context (drivers)	The application needs to run on the user's computer and access local files. CON-01 requires the application to run locally rather than relying on an external server.
Options considered	1. Host the application on an external server and access it remotely. 2. Run the application locally using Java and command line tools.
Decision	Run the application locally as a Java application that communicates with WSL to execute Linux commands.
Consequences	Favors local accessibility and allows the application to work with files on the user's computer. The accepted cost is that users must have Java and WSL installed and properly configured.
Evidence	CON-01 establishes the requirement for local execution. Section 6.3 identifies the Java GUI and application components, while Section 6.4 shows that the application runs on the user's PC.
Related	CON-01, CON-03, FR-02, R1

ADR-03 [Read-Only File System Access]

Status	Accepted
Date and owner	11/06/2026 - George (Drafted)
Context (drivers)	The application needs to allow users to browse folders and search for files without modifying their existing data. CON-02 requires the application not to write, move, or delete files.
Options considered	1. Allow users to view and modify files. 2. Restrict the application to read-only file access.
Decision	Restrict the application to read-only access, allowing users to browse folders, view file information, and search for files without modifying them.
Consequences	Favors data safety and prevents accidental modifications to user files. The accepted cost is that users cannot create, rename, move, or delete files through the application.

Evidence	CON-02 establishes the read-only restriction. The project scope also excludes file movement and other file-modification features.
Related	CON-02, FR-01, FR-02, FR-03, FR-07, FR-08

ADR-04 [Using WSL Linux Commands]

Status	Rejected
Date and owner	11/07/2026 - George (Drafted), Austin
Context (drivers)	The application needs to search for files and retrieve folder information efficiently. CON-03 establishes the use of Linux commands through WSL.
Options considered	1. Use native Windows commands for searching and retrieving file information. 2. Use Linux commands through WSL, such as grep and ls. 3. Use a mix of command line tools to achieve best results.
Decision	3. Use a mix of command line tools for best results.
Consequences	Favors performance and supports file searching and folder retrieval. The accepted cost is that users must have Java, WSL, and Windows installed and configured.
Evidence	App prototype currently uses java commands only. S-01 uses powershell to parse file data.
Related	QR-01, FR-03, CON-03, R2

7.2 Trade-off summary

Quality favored	Architectural move	Cost accepted	Recorded in
Maintainability	Using MVC architecture to separate presentation, application control, and backend operations.	Additional coordination between components and the need to maintain clear interfaces.	ADR-01

Local accessibility	Running the application locally using Java.	Users must have Java and command line access installed and configured.	ADR-02
Data safety	Restricting the application to read-only file access.	Users cannot create, rename, move, or delete files through the application.	ADR-03
Performance	Using Java and command line for searching and retrieving file information.	Users must have Java installed and configured.	ADR-04

8 Prototype Evidence: Architecture Spike

Spike ID and title	S-01: ls parsing
Risk or decision it addresses	R1,R2 / ADR-01
Question (answerable yes or no)	Can the application parse command output to retrieve folder and file information?
Set-up	Write a script to collect file information and export it as data that can be read by the application. Record timestamps of script execution and completion.
How it was measured	Tested whether the command could execute successfully and whether its output could be collected and interpreted by the application.
Result (numbers)	Powershell script displays working directory's children with details. Executes within 1 second.

What it changed or confirmed	Supports the decision to use WSL Linux commands (ADR-01) for retrieving file and folder information.
Where to see it	/spikes/s-01/

Spike ID and title	S-02: WFE search vs wsl find
Risk or decision it addresses	R2 / ADR-01
Question (answerable yes or no)	Are linux file searches faster than windows file explorer?
Set-up	Both apps will be directed to the root of a drive, in this case Q:, and be searched for the string "3354" at 00 seconds. Completion times will be captured for both apps. Compare search and time results between the two.
How it was measured	Starting from 00, seconds count on pc clock will be screen captured when search completes
Result (numbers)	WFE: 39 seconds / WSL: 12 seconds
What it changed or confirmed	Confirms searching for strings in filenames is faster in linux machines than windows. Additionally, WFE search was much more incomplete, containing only 14 results to WSL's 223.

Where to see it	/spikes/s-02/

9 Traceability

9.1 Extended traceability matrix

Requirement	Use case	Analysis classes	Component(s)	QAS / ADR	Evidence

FR-01	UC-01: View Files	File, File system	GUI Presentation, File Explorer, WSL Translator	ADR-01, ADR-02, ADR-03	UC-01 analysis; ls -l acceptance test S-01
FR-02	UC-02: View Folders	Folder, File system	GUI Presentation, File Explorer, WSL Translator	ADR-01, ADR-02, ADR-03, ADR-04	UC-02: use-case analysis; S-01
FR-03	UC-03: Search Terms	File, Folder, Search Results	Search, WSL Translator	QAS-01, ADR-01, ADR-04	UC-03: analysis; P1 grep feasibility test
FR-04	No dedicated use case expanded in Section 4.2	Quick Access Folder	Sidebar, GUI Presentation	ADR-01	FR-04 acceptance criteria
Requirement	**Use case**	**Analysis classes**	**Component(s)**	**QAS / ADR**	**Evidence**
QR-01	UC-03: Search Terms	Search Results, File, Folder	Search, WSL Translator	QAS-01, ADR-01	P1 grep feasibility test

QR-02	UC-02: View Folders	Folder, File system	File Explorer, WSL Translator	ADR-01, ADR-04	UC-02 analysis
CON-01	UC-02, UC-03	File system, Search Results	GUI Presentation, File Explorer, Search, Sidebar, WSL Translator	ADR-02	Local Java GUI implementation

9.2 Gap analysis

Gap	Items	What you did about it
Requirements with no component	FR-04 has no dedicated usecase, but has a component assignment through Sidebar and GUI Presentation.	Identified Sidebar and GUI Presentation as the components responsible for quick access functionality.
Components with no requirement, QAS or ADR	Component-to-requirement mappings needed to be checked against the completed Section 6.3 architecture.	Reviewed component responsibilities and connected them to relevant requirements, quality scenarios, and architecture decisions.
Use cases not analyzed (and why that is acceptable)	None. UC-01, UC-02, and UC03 have all been expanded in Section 4.2.	All three selected use cases were analyzed because they represent the main file viewing, folder navigation, and search functions.

10 Project Plan (updated; was P1 section 4)

10.1 Roles and working agreement

[Keep your P1 text and record only what changed. If roles moved between members, say so here and in 0.2.]

10.2 Plan to P3

Milestone	Date	Owner	Done when

Finalize P2 architecture and design	P2 submission date	Entire team	P2 documentation is reviewed and submitted.
Implement core application functionality	Team-agreed date	Assigned developers	File viewing, folder navigation, quick access, and search are implemented.
Integrate components and perform testing	Team-agreed date	Entire team	Components work together and acceptance tests are completed.
Finalize P3 deliverables	P3 submission date	Entire team	Implementation, documentation, testing evidence, and required deliverables are complete.

10.3 Risks (updated)

Risk	Status since P1	Likelihood / impact	Mitigation and owner
R1: Team member(s) unable to get WSL and Java working on their machine.	Unresolved	Low / Low	Ashton checks that every member has WSL installed and working on their computer.
R2: Calling Linux commands from Java through WSL turns out slower than anticipated.	Answered	Medium / Medium	Austin runs a trial between searching "3354" in the file explorer and running "find . name '3354'" in wsl, then

Risk	Status since P1	Likelihood / impact	Mitigation and owner

			compares the results to see which is faster.

R3: External commitments like other classes or jobs leaving a person(s) unable to deliver their work by the deadlines.	Closed	Medium / High	Zac Gregory checks on everyone's individual progress on days 10/28 and 11/1, names backups for each task, and reassigns work if something is more than 2 days behind schedule.
R4 (updated): Requirements change informally.	Resolved	Medium / Medium	Austin documents research and reasoning for requirement changes and verifies sanity across traceability matrix and other requirements.
R5: The design is built on an unresolved premise: Q1 and Q2 are open.	Resolved	Medium / High	Yukai Ma gets answers or a team decision recorded in section 5 by 10/20, and any change to the functional requirements or constraints goes through the revision history.
R6 (new): It is not possible parse linux commands into workable data for the GUI	Resolved	Medium / Medium	Ashton finds other methods of file/folder lookup for the GUI.

10.4 Repository

https://github.com/AshtonElliott/SimpleFileExplorer

- '/p2' contains diagrams and documents for P2 of the project
- '/spikes' contains documentation of spikes performed for the project, organized in folders named '/s-0#'
- Commits and PRs must have relevant titles describing their changes

10.5 Use of AI tools in P2

Member	Tool	What it was used for	What you checked or changed

Austin Robson	Claude	Converting P2 template pdf into markdown	Verify that formatting such as tables and headings match up with original pdf
Ashton Elliott	Claude Opus 5.5	Assisted in creation of Search Engine to app.java	Verified that the code worked as expected as well as added additional comments for team members to be able to identify what happens where in the code.
George Chakiath Dani	ChatGPT	Adding in information for information Sections 1-3 and also including new information for Section 4: Use Case Analysis	Helped give me ideas on how I could fill in each of the blank spaces based on information in P1 and used it accordingly to the instructions at hand.
George Chakiath Dani	ChatGPT	Assisted with drafting ADR-01, ADR-02, ADR-03, and the Section 72 - Tradeoff Summary	Reviewed and edited the proposed decisions, checked them against project constraints and requirements, and adjusted wording to reflect the team's actual decisions and approval status.
George Chakiath Dani	ChatGPT	Assisted with drafting Section 8 - Prototype Evidence: Architecture Spike.	Reviewed the spike description, checked its connection to ADR-01 and R2, and clarified that missing measurement and test evidence should not be fabricated
George Chakiath Dani	ChatGPT	Assisted with drafting Section 9 - Traceability Matrix and Gap Analysis	Reviewed requirement-to-usecase mappings, analysis classes, ADR references, and identified gaps requiring confirmation in Section 6
George Chakiath Dani	ChatGPT	Section 7: helped improve the wording, structure, and clarity of architecture	Reviewed the decisions, alternatives, consequences, and related requirements to ensure

Member	Tool	What it was used for	What you checked or changed
		decision records and the trade-off summary.	they reflected the project's architecture
George Chakiath Dani	ChatGPT	Section 8: Helped improve the wording and organization of the architecture spike descriptions.	Checked the descriptions against the spike information and kept unrecorded results and missing evidence identified as pending.
George Chakiath Dani	ChatGPT	Section 9: Helped review and improve the traceability analysis.	Checked the mappings against the project's requirements, use cases, analysis classes, components, and architecture decisions.
George Chakiath Dani	ChatGPT	Section 0.2: Helped organize and improve the wording of changes made since the P1 baseline.	Reviewed the changes against the actual P2 work and identified items requiring team confirmation.
George Chakiath Dani	ChatGPT	Section 10: Helped organize and refine the P3 milestone plan and completion criteria.	Reviewed the milestones to ensure they reflect the project's planned implementation, integration, testing, and submission work.
George Chakiath Dani	ChatGPT	Section 12: Helped clarify the baseline declaration and organize the team signature table	Checked that team members, signatures or initials, and dates were represented appropriately.
Yukai	Gemini AI	Use Gemini to search for the information about component diagram, also use gemini AI to get some basic ideas of component's responsibility	I changed the component diagram and the responsibilities of the components.

11 Open Questions (updated; was P1 section 5)

ID	Question	Raised by	Owner and due date	Status or answer	Assumption meanwhile
Q1	As the file explorer needs to run on Linux in Windows, why don't we just use Windows local commands?	Yukai Ma	Austin Robson - 10/18	Windows commands alone tend to be slow. Dedicated tools, such as bash, java, or even powershell, can be used to achieve the same behavior with better speed.	Users will only be able to use the app if linux subsystem is installed
Q2	Do we need to add a tutorial to teach users to download WSL before using this file explorer?	Yukai Ma	Austin Robson - 10/18	Windows susbsystem linux, and/or any other dependencies, will be included in the app README.md with intallation instructions	Users will have to learn how to install WSL themselves

12 Baseline v1.0 Declaration (was P1 section 6)

We declare this document, version 1.0, the baseline for P2. Sections 1 to 9 describe what we will build in P3. Any later change to them is recorded in section 0.2 with its reason.

Team member	Signature or initials	Date
Austin Robson	AR	11/8/26
George Chakiath Dani	GCD	11/8/26
Ashton Elliot	AEE	11/8/26
Yukai Ma	YM	11/8/26
Rowan Egan		

Zachary Gregory	ZG	11/8/2026